Lesson 9 Teacher's Key: On the Scoreboard

GDevelop - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

That a persistent HUD in GDevelop needs one small addition beyond what score/lives already needed: an explicit "always update the display" event, since Text objects don't automatically reflect variable changes the way Scratch's watchers or 001GC's real-time Fields do.

Level 1 - Full Worked Solution

- Score is a new scene variable, parallel to EnemiesRemaining and Lives.
- The Score = Score + 10 action lives in the Combat group's collision event - the exact same event that already changes EnemiesRemaining, since both are consequences of the exact same confirmed hit.
- The new always-event (no condition) is what makes this a genuinely continuous HUD - GDevelop's Text objects hold whatever text they were last explicitly set to, so this event needs to run every frame to stay accurate.

Highlighted lines are new this week; everything else is code you already have.

```
(Combat event group)
Condition: Bullet is in collision with Enemy
Action: Play a sound (sfx_hit.wav)
Action: EnemiesRemaining = EnemiesRemaining - 1
Action: Score = Score + 10
Action: Delete Bullet
Action: Delete Enemy

(new event, always - no condition)
Action: Change text of ScoreText: "Score: " + ToString(Score)
Action: Change text of LivesText: "Lives: " + ToString(Lives)
```

Why This Structure

- This is a useful contrast for students moving between engines this week: GDevelop's Text objects are 'dumb' displays that need to be told what to say, unlike Scratch's watchers (always live) or 001GC's real-time Fields (bound automatically) - worth naming this difference explicitly.
- Keeping the HUD-update event unconditional (no condition at all) guarantees the display never falls behind, at the small, harmless cost of re-setting text that sometimes hasn't actually changed.

Level 2 / Level 3 Talking Points

- Ask students to predict what would happen if the HUD-update event had a condition attached (e.g., only running on a collision) - the display would only refresh at those specific moments, which happens to work for score/lives here but would break for anything that changes for other reasons (like a countdown timer).
- Level 3 structure: this is a good moment to introduce or reinforce GDevelop's event groups as a scaling technique - a project with Combat, Spawn, Movement, and now HUD groups is starting to look like a real, organized project.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
never score;
// both numbers are correct the instant the game starts, not only after
the first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
(Combat event group - Level 3)
Condition: Bullet is in collision with Enemy
Action: Play a sound (sfx_hit.wav)
Action: EnemiesRemaining = EnemiesRemaining - 1
Action: Score = Score + (5 + Random(10))
Action: Delete Bullet
Action: Delete Enemy

(HUD group - Level 3: move the always event into its own event group named
HUD)
Action: Change text of ScoreText: "Score: " + ToString(Score)
Action: Change text of LivesText: "Lives: " + ToString(Lives)
(font and color for ScoreText/LivesText are set in each object's editor)
```

Common Student Mistakes

- Forgetting the always-event entirely and expecting the Text objects to update automatically just because the Score variable changed - they won't, GDevelop requires an explicit Change Text action.
- Adding Score = Score + 10 to the wrong event (escape-cleanup instead of Combat).
- ScoreText and LivesText positioned at the same coordinates, causing visual overlap.